Le programme ci-dessous est implémenté dans une machine à café dont les boutons envoies les signaux CAFE, CREME et ANUL au programme principal (c.f. code).

1. Combien de processus ce programme génère-t-il ? Décrire le rôle et les actions de chaque processus.
   //Ce programme créer un processus enfant à la fois car si un nouveau signal sig_cafe est envoyé alors qu'un autre est déjà en cours de //traitement dans son handler, celui-ci est automatiquement masqué jusqu'à la fin de l'exécution du handler en cours.
2. Que ce passe-t-il si le bouton CAFE est pressé plusieurs fois de suite ?
   //Un signal reçu est automatiquement masqué quand il est en train de gérer son handler, il ne peut
   //donc n'y avoir qu'un café en attente
3. Que ce passe-t-il si l'on appuye sur SIG_CREME sans qu'un café ne soit lancé ? Que ce passe-t-il si l'on appuye sur SIG_CREME lorsqu'un café est lancé ?
   //Si aucun café n'est lancé et on appuie sur SIG_CREME, le signal sera ignoré.
   //Lorsque qu'un café est lancé et on se trouve dans le processus enfant, le signal SIG_CREME va être associé au service_handler, si on est dans le parent, celui-ci transmet à l'enfant.
4. Dans quel cas le programme affiche-t-il l'erreur "Erreure survenue, cafe interrompu" ?
   //Dans le cas ou le processus enfant ne se termine pas correctement
5. Pourquoi utiliser la fonction "write" au lieu de "printf" dans les handlers de signaux ?
   //printf créer des effets de bord tandis que write non. En cas de réception d'un signal comme kill il n'y a donc pas de risque de perte de zone mémoire
   //par exemple

cafe.c

```c
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>
//Directives de précompilation
#define SIG_CAFE SIGUSR1 //Gestion du signal défini par l'utilisateur
#define SIG_CREME SIGUSR2
#define SIG_ANUL SIGINT //Interruption du processus ctr+C
//Pointeur non modifiable et constante non modifiable
const char * const cremeStr = "Ajout de creme sur le cafe\n";
const char * const anulStr = "Cafe interrompu par l'utilisateur\n";
const char * const errStr = "Erreure survenue, cafe interrompu\n";

pid_t pidService = 0;//Variable d'identification du processus

void exit_err(const char* str) //Fonction qui affiche un message d'erreur errno
{                                //et quitte le programme avec le code d'erreur
    perror(str);                 //EXIT_FAILURE
    exit(EXIT_FAILURE);
}
 //Fonction qui associe une action à un signal
void set_sigaction_handler(int signum, void (*sig_handler)(int))
//Le parametre sig_handler est une fonction "handler" ou SIG_IGN
{
    struct sigaction action;//Structure de type sigaction

    if(sigemptyset(&action.sa_mask) == -1)
        exit_err("set_sigaction_handler, sigemptyset");
                                                //Initialise l'ensemble des signaux donnés
    action.sa_flags = 0;                        //par action.sa_mask à vide (garanti
    action.sa_handler = sig_handler;            //qu'aucun signal n'est masqué)
    if(sigaction(signum, &action, NULL) == -1)
```

//sa_flags mis à 0 car on n'utilise pas la fonction sa_sigaction (SA_SIGINFO au
//lieu de 0) mais sa_handler
//sa_handler est la fonction qui va être utilisée pour gérer le signal
//sigaction defini l'action du signal signum avec l'action et la config initiale si il y en a une

```
        exit_err("set_sigaction_handler, sigaction");
}

void service_handler(int sig) //Gestion d'un handler de type service avec sig le numero
{                             //de signal
    switch(sig)
    {
        //SIG_CREME recu
        case SIG_CREME:
            write(STDIN_FILENO, cremeStr, strlen(cremeStr));
            break;

        //SIG_ANUL recu
        case SIG_ANUL:
            write(STDIN_FILENO, anulStr, strlen(anulStr));
            exit(EXIT_SUCCESS);
            break;

        default:
            fprintf(stderr, "Signal %d non attendu\n", sig);
            exit(EXIT_FAILURE);
    }
}

void service()
{
    set_sigaction_handler(SIG_CREME, service_handler);//Associe le handler de service
    set_sigaction_handler(SIG_ANUL, service_handler); //au signal SIG_CREME et
                                                      //au signal SIG_ANUL
    int err;
    sigset_t set, oldset;//Déclaration de deux variables de type sigset_t
    err = sigemptyset(&set);//Initialisation de set à vide
    err |= sigaddset(&set, SIG_CREME);//Ajoute le signal SIG_CREME à l'ensemble set
    if(err != 0)
        exit_err("service, construction d'ensemble de signaux");
    if(sigprocmask(SIG_BLOCK, &set, &oldset) == -1)//Fonction de masquage SIG_BLOCK
        exit_err("service, sigprocmask 1");        //ajoute le set à la liste des signaux déjà
                                                   //bloqués.
    printf("Je commence a preparer un cafe\n");    //oldset est la liste des signaux
    sleep(5); //Met le processus en veille         //initialement masqués
              //pendant 5 secondes
    if(sigprocmask(SIG_SETMASK, &oldset, NULL) == -1)//Les signaux actuellement
        exit_err("service, sigprocmask 2");          bloqués sont remplacés par oldset

    printf("Cafe termine\n");
    exit(EXIT_SUCCESS);
}

void btn_handler(int sig)//Gestion d'un handler de type btn
{
    switch(sig)
    {
        //SIG_CAFE recu
        case SIG_CAFE:
            pidService = fork();Création d'un processus enfant
```

```c
            if(pidService == -1)
                exit_err("btn_handler, fork");
            else if (pidService > 0)//Processus parent
            {
                set_sigaction_handler(SIG_CREME, btn_handler);//Associe le handler de btn
                set_sigaction_handler(SIG_ANUL, btn_handler); //au signal SIG_CREME et
                                                              //au signal SIG_ANUL
                int status = 0;
                while( (status = waitpid(pidService, NULL, 0)) == -1)
                    if(errno != EINTR)//Attente du processus enfant pidService
                        exit_err("btn_handler, waitpid");
                if(WIFEXITED(status))//Retourne True si l'enfant s'est terminé correctement
                    if(WEXITSTATUS(status) != 0)//Retourne le statut de l'enfant
                        write(STDIN_FILENO, errStr, strlen(errStr));
            }
            else
                service();//Processus enfant, on appel la fonction service
            break;

        //SIG_CREME recu
        case SIG_CREME:
            if(kill(pidService, SIG_CREME) == 1)//Envoie le signal SIG_CREME au
                exit_err("btn_handler, kill");   //processus pidService
            break;

        //SIG_ANUL recu
        case SIG_ANUL:
            if(kill(pidService, SIG_ANUL) == 1)//Envoie le signal SIG_ANUL au
                exit_err("btn_handler, kill"); //processus pidService
            break;

        default:
            fprintf(stderr, "Le signal %d non attendu\n", sig);
            exit(EXIT_FAILURE);
    }
}

int main(void)
{
    set_sigaction_handler(SIG_CAFE, btn_handler);Création des actions de signaux
    set_sigaction_handler(SIG_CREME, SIG_IGN);
    set_sigaction_handler(SIG_ANUL, SIG_IGN);

    while(1)
        pause(); //Boucle d'attente de signal

    return 0;
}
```

Makefile

```makefile
all: cafe //Ensemble des exécutables à produire
                //l'option -g génère des infos supplémentaire pour le debogage et -Wall active tous
cafe: cafe.c //les warnings possibles
        gcc cafe.c -g -Wall -o cafe
```